SESSION 04 / PRINTABLE GUIDE

Lesson 6: Use Data Wrangler to discover and review transformations

Session 04 | Guided visual data preparation
Output: An inspected sample, a reviewed generated PySpark function, and full-batch checks before persistence.

Tier choice: The code/counts below explain the nine-row micro fixture. For the primary lesson, use the main lab's Wrangler step: a labeled sample for discovery, followed by applying the reviewed export to all 201,579 rows. The UI/export principles here apply to both tiers; do not copy the micro-fixture batch filter or nine-row assertion into the main lab.

What Data Wrangler does

Data Wrangler provides an interactive profile and cleaning interface inside Fabric notebooks. It can work with pandas and Spark DataFrames. For a Spark input, the documented experience temporarily uses a pandas sample for exploration and translates supported transformations back to PySpark when exporting to the notebook.

That is useful for teaching the connection between an operation such as trimming a column and the code that implements it. It is not a substitute for approving a business rule or validating all records.

The key sequence is:

> Open a DataFrame -> inspect a labeled sample -> preview an operation and its code -> apply/discard -> export -> review the PySpark -> run it -> verify full-batch results -> persist only an approved contract.

Prepare the demonstration

Run Cell 1 of the device example. Keep the original raw file and Bronze table unchanged. In a separate exploration cell:

```python
from pyspark.sql import functions as F

wrangler_input = (
    spark.table("s04_device_events_bronze")
    .filter(F.col("batch_id") == "s04-devices-001")
)
display(wrangler_input.limit(9))
```

Wait until the notebook kernel is idle. From the Home ribbon's Data Wrangler dropdown, choose the active Spark DataFrame `wrangler_input`. If the tenant exposes a cell-level Open in Data Wrangler entry, it is another way to enter the same workflow.

Use a custom sample covering all nine fixture rows. Record the sample size/method. For a large real table, do not assume the default preview includes rare invalid values, later schema variants, or cross-partition duplicates.

A Spark input does not require manually collecting the entire source with `toPandas()`. Let the verified UI manage its preview sample; the exported function must be applied to the intended full Spark DataFrame.

Walk through the profile

Select columns and inspect the Summary panel and column headers:

Column	What to notice	Teaching question
`device_id`	Spaces/casing create three spellings for two devices	Is a spelling change a new device?
`event_id`	E002 is repeated for D01	Which columns define the event key?
`temperature_c`	String data contains `sensor error` and 200.00	Is present data necessarily usable numeric data?
`firmware_version`	2.1/2.2 look numeric but are version labels	Would converting version text to a decimal preserve its meaning?
`signal_strength_dbm`	Numeric diagnostic context arrives as text	Is a technical field always noise?
`ui_theme`	All values are `dark` in this fixture	Is it excluded because it is constant, or because it has no analytical role?
`export_row_number`	Unique per file, not a business event key	Could this uniqueness hide a duplicated event?

Column visuals depend on type. Do not force the entire invalid temperature column to numeric just to obtain a histogram, then silently drop the conversion errors. Use the explicit safe-conversion and quarantine rules from the notebook lesson.

Preview a small, explainable cleaning recipe

Use the operations available in the actual tenant. For this demonstration:

1. Strip whitespace from `device_id` and `event_id`.
2. Convert those two identifiers to uppercase under the teaching key convention.
3. Rename `temperature_c` to `temperature_text` to distinguish raw text from a later typed measure.
4. Change `signal_strength_dbm` to an integer after checking that all nine fixture values parse.
5. Drop `ui_theme`, `debug_message`, and `export_row_number` from the candidate analytical DataFrame, not from Bronze.

For each step, pause at the generated-code preview. Ask what the operation changes, what evidence it could destroy, and whether it is justified by the contract. Apply a justified step or discard it.

Do not demonstrate "drop duplicates" as the solution to E002. The approved policy quarantines both occurrences. Do not fill an invalid temperature with zero or the mean. Do not use Flash Fill to invent an event identifier.

Dropping a column here is a scoped analytical choice, not a source deletion or retention policy. The final Silver implementation uses an allowlist, so a newly arriving field cannot silently leak into the serving contract.

Export, review, and run

Export the cleaning steps back to the notebook as a function. For a Spark input, inspect the translated **PySpark** preview and the function actually generated. A pandas export is not interchangeable with a Spark function.

The export adds code; it does not execute the transformation or overwrite the original DataFrame/table. Run the new cell deliberately, apply the function to the full `wrangler_input`, and assign the returned result to a new name.

The function name and exact generated expressions can vary. The following is a manually reviewed equivalent of the intended operations, not a claim that the UI emits these exact lines:

```python
def reviewed_wrangler_candidate(df):
    return (
        df.withColumn("device_id", F.upper(F.trim("device_id")))
```

```
        .withColumn("event_id", F.upper(F.trim("event_id")))
        .withColumnRenamed("temperature_c", "temperature_text")
        .withColumn(
            "signal_strength_dbm",
            F.expr("try_cast(signal_strength_dbm as int)"),
        )
        .drop("ui_theme", "debug_message", "export_row_number")
    )


wrangler_candidate = reviewed_wrangler_candidate(wrangler_input)

if wrangler_input.count() != 9 or wrangler_candidate.count() != 9:
    raise ValueError("This recipe must preserve all nine source event rows.")

wrangler_candidate.groupBy("device_id", "event_id").count().filter(
    F.col("count") > 1
).show()

wrangler_candidate.select(
    "device_id", "event_id", "temperature_text",
    "firmware_version", "signal_strength_dbm",
).show(truncate=False)
```

Expect nine candidate rows, with D01/E002 still occurring twice and `sensor error` still visible. These transformations improve representation; they have not performed complete validation or produced accepted Silver.

For a real export, apply the generated function using its actual name and verify these same invariants. Test how its casts handle invalid/null inputs. Do not assume the displayed sample validates those branches.

Return to the accepted Silver workflow

Resume the device notebook at Cell 2, which reads Bronze and implements the approved quality rules. The Wrangler candidate is a sandbox and is not substituted into that code: the source names and quality expectations must remain consistent.

When developing a production notebook, integrate reviewed generated steps into the versioned transformation once, in the correct order. Do not run both an interactive experiment and an unrelated copy of the logic and assume they are equivalent.

Then verify the actual accepted/quarantine outputs: four accepted events, five rejected rows, and two daily summaries. Persist the explicit Silver allowlist, not the whole Wrangler working DataFrame.

Limitations and recovery

- Spark preview statistics describe the chosen sample; full-data Spark checks establish full-batch conditions.

- Custom-code operations inside Data Wrangler are documented for pandas only. Keep custom Spark rules in normal notebook cells.
- A notebook must be idle to launch the UI. UI layout and available entries can vary.
- Verify Data Wrangler and Spark support in the actual GCC High tenant. General documentation is not a feature-availability guarantee.
- If unavailable, label the UI demonstration blocked or use approved captured evidence. The ordinary PySpark walkthrough still teaches the transformations, but does not prove Data Wrangler execution.

Exercise

Have learners record a sample size, three justified cleaning steps, one operation they rejected, the reviewed PySpark function location, and the full-batch row/duplicate checks.

Ask: "Why did we keep firmware and signal strength while excluding UI theme?" A strong answer cites future diagnostic use and the layer contract, not simply whether a column is numeric, constant, or technical.

Next: Silver columns and Gold processing.

References, reviewed October 29, 2026: Data Wrangler overview and Data Wrangler with Spark.

Source: 06_data_wrangler.md. Companion-material links open the repository originals.